AUSBILDUNGSUNTERLAGE

AVR-Programmierung mit dem ATmega32

Einstiegsanleitung für Windows 11

Controller	**ATmega32 / ATmega32A**
Trägerboard	**ZIF-Sockel, 8-MHz-Quarz**
Programmiergerät	**DIAMEX PROG-S (STK500v2)**
Entwicklungsumgebung	**CLion mit CMake**
Sprache	**C (C99)**

Inhalt

Teil 0 — Worum es geht

Teil 1 — Deine Hardware

Teil 2 — Software installieren

Teil 3 — Programmiergerät anschließen

Teil 4 — Erster Kontakt zum Controller

Teil 5 — Die Fuses einstellen

Teil 6 — Das CLion-Projekt

Teil 7 — Das erste Programm

Teil 8 — Bauen

Teil 9 — Flashen

Teil 10 — Ausgabe am PC ansehen

Teil 11 — Wenn etwas nicht klappt

Anhang A — Glossar

Anhang B — Befehlsübersicht

Anhang C — Checkliste

Teil 0 — Worum es geht

Was ist ein Mikrocontroller?

Ein Mikrocontroller ist ein vollständiger kleiner Computer in einem einzigen Chip: Rechenwerk, Speicher und Anschlüsse für die Außenwelt. Der ATmega32, mit dem du arbeiten wirst, hat unter anderem:

- **32 KB Flash-Speicher** für dein Programm
- **2 KB RAM** für Variablen während der Laufzeit
- **1 KB EEPROM** für Daten, die einen Stromausfall überleben sollen
- **32 Anschlüsse**, die du als Ein- oder Ausgänge schalten kannst

Anders als bei deinem PC gibt es kein Betriebssystem. Dein Programm ist das Einzige, was läuft. Es startet, sobald Spannung anliegt, und läuft bis zum Abschalten.

Wie kommt das Programm in den Chip?

Fünf Schritte, die du gleich in der Praxis durchläufst:

```
main.c              Du schreibst C-Code
   │
   │  Compiler (avr-gcc) übersetzt in Maschinencode
   ▼
hello.elf           Programmdatei mit Zusatzinformationen
   │
   │  avr-objcopy schneidet den reinen Maschinencode heraus
   ▼
hello.hex           Das, was in den Chip soll
   │
   │  avrdude schickt es über das Programmiergerät
   ▼
ATmega32            Programm liegt im Flash und läuft
```

Die Werkzeuge `avr-gcc`, `avr-objcopy` und `avrdude` sind Kommandozeilenprogramme. Du wirst sie nicht jedes Mal von Hand aufrufen — CLion erledigt das für dich. Aber du solltest wissen, dass sie existieren, weil Fehlermeldungen von ihnen kommen.

Was ist eine Toolchain?

Die Sammlung aller Werkzeuge, die aus deinem C-Code ein lauffähiges Programm für den Chip machen. „AVR-Toolchain" heißt: Compiler und Hilfsprogramme, die nicht für deinen PC übersetzen, sondern für den ATmega. Das nennt man **Cross-Compiling**.

Begriffe, die dir dauernd begegnen werden

Begriff	Bedeutung
Flashen	Das Programm in den Programmspeicher des Chips schreiben
ISP	*In-System-Programming* — Flashen, während der Chip eingebaut bleibt
Fuse	Dauerhafte Grundeinstellung im Chip, z. B. welche Taktquelle er benutzt
F_CPU	Die Taktfrequenz, mit der der Chip läuft. Bei euch **8 MHz**
UART	Serielle Schnittstelle, über die der Chip Text an den PC schicken kann
Baudrate	Übertragungsgeschwindigkeit des UART. Bei uns **9600**

Ein ausführlicheres Glossar steht in Anhang A.

So liest du diese Anleitung

Nach jedem Arbeitsschritt findest du einen Kasten wie diesen:

> **✓ Kontrolle**
>
> Hier steht, was passiert sein muss, bevor du weitermachst.

Mach erst weiter, wenn die Kontrolle stimmt. Wenn du drei Schritte übergehst und es dann nicht funktioniert, suchst du den Fehler an drei Stellen gleichzeitig. Das ist der häufigste Grund, warum Leute an dieser Stelle verzweifeln.

Teil 1 — Deine Hardware

Was auf dem Tisch liegt

Teil	Beschreibung
Trägerboard	Fertig bestückte Platine mit einem Sockel für den Chip
ATmega32	Der Controller, ein schwarzer Baustein mit 40 Beinen
PROG-S	Das Programmiergerät, kommt per USB an den PC
USB-Seriell-Adapter	Damit der Chip später Text an den PC schicken kann
USB-Kabel, Drahtbrücken	

Auf dem Board ist bereits alles verdrahtet, was der Chip zum Laufen braucht: Stromversorgung, Abblockkondensatoren, ein Widerstand am Reset-Anschluss und der **Quarz**.

Der Quarz

Der Quarz ist das kleine glänzende Metallgehäuse neben dem Sockel. Er gibt dem Chip seinen Takt — eine Art Herzschlag, der bestimmt, wie schnell der Controller arbeitet.

Auf euren Boards sitzt ein 8-MHz-Quarz. Acht Millionen Schwingungen pro Sekunde. Diese Zahl brauchst du später an zwei Stellen, merke sie dir:

- In der Projektdatei `CMakeLists.txt`, als `F_CPU = 8000000UL` (siehe Teil 7.1)
- Bei den Fuses, damit der Chip den Quarz überhaupt benutzt

Wenn du auf dem Quarzgehäuse nachsiehst, steht dort `8.000` oder `8.000 MHz`.

Der Sockel mit Hebel

Der Chip sitzt in einem **ZIF-Sockel** (*Zero Insertion Force*, „ohne Kraftaufwand"). Der Hebel an der Seite öffnet und schließt die Kontakte, sodass du den Chip wechseln kannst, ohne ihn zu beschädigen.

So wechselst du einen Chip

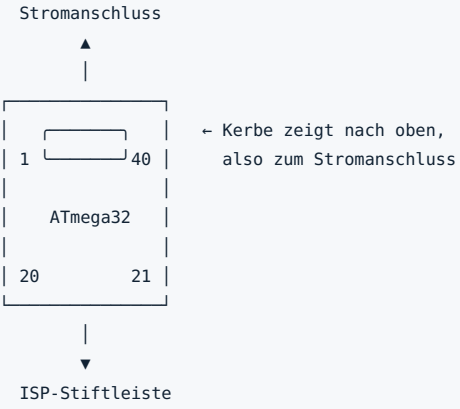
1. **Strom weg.** Programmiergerät abziehen, Netzteil trennen. Nicht bei anliegender Spannung wechseln.
2. **Hebel hochstellen.** Er steht dann senkrecht, die Kontakte im Sockel sind offen.
3. **Alten Chip gerade herausheben.** Nicht verkanten.
4. **Neuen Chip einsetzen.** Auf die **Kerbe** achten — sie zeigt zum Stromanschluss, siehe unten.
5. **Hebel herunterdrücken**, bis er einrastet.
6. **Erst jetzt** Strom anlegen.

Die Kerbe — in welche Richtung?

An einem Ende des Chips ist eine halbrunde Kerbe eingeprägt. Sie markiert das Ende, an dem Pin 1 liegt.

Auf **eurem** Board gilt:

> **Die Kerbe zeigt zum STROMANSCHLUSS.**
>
> **Nicht zur ISP-Stiftleiste!**



Dreh das Board so hin, dass der Stromanschluss oben liegt. Dann muss die Kerbe des Chips ebenfalls nach oben zeigen.

> **ACHTUNG** **Setzt du den Chip andersherum ein und schaltest den Strom an, ist er praktisch immer zerstört.** Dann liegen Versorgungsspannung und Masse vertauscht an, und der Chip erhitzt sich innerhalb von Sekunden.
>
> **Kontrolliere die Kerbe zweimal:** einmal, bevor du den Hebel schließt, und noch einmal, bevor du den Strom anlegst. Das kostet fünf Sekunden und spart im Zweifel einen Chip.
>
> **Statische Aufladung.** Fass vor dem Anfassen loser Chips kurz an ein geerdetes Metallteil, zum Beispiel ein Gehäuse eines eingesteckten Geräts. Elektronik verträgt die Entladung aus einem Pulloverärmel nicht immer.

Die Anschlüsse am Board

Du musst nur zwei Dinge anschließen.

1. Das Programmiergerät

Es kommt an die **ISP-Stiftleiste** des Boards. Die hat entweder 6 oder 10 Stifte. Wenn beide vorhanden sind, nimm die 6-polige — sie lässt sich schwerer falsch aufstecken.

Am Stecker gibt es eine Markierung für Pin 1, oft ein kleines Dreieck oder eine farbige Ader am Flachbandkabel. Diese Markierung zeigt zur Pin-1-Markierung auf der Platine.

2. Der Seriell-Adapter

Der wird erst in Teil 10 gebraucht. Drei Drähte:

Adapter	Board / Chip
TX	Pin 14 (PD0 / RXD)
RX	Pin 15 (PD1 / TXD)
GND	GND

> **TIPP** **TX und RX werden gekreuzt.** Was das eine Gerät sendet (*Transmit*), muss das andere empfangen (*Receive*). TX an TX zu stecken ist der mit Abstand häufigste Anfängerfehler bei serieller Verbindung.

Viele Boards führen PD0, PD1 und GND schon auf eine eigene kleine Stiftleiste. Wenn deines das hat, nimm die — dann musst du nicht am Sockel herumstochern.

> **✓ Kontrolle**
>
> - Der Chip sitzt im Sockel, die **Kerbe zeigt zum Stromanschluss**, der Hebel ist eingerastet.
> - Du hast auf dem Quarz nachgesehen und dort steht 8 MHz.
> - Du weißt, wo die ISP-Stiftleiste sitzt.

Teil 2 — Software installieren

Drei Dinge werden installiert: die AVR-Toolchain, das Programm avrdude und CLion.

2.1 — PowerShell öffnen

Drücke die **Windows-Taste**, tippe `PowerShell` und öffne **Windows PowerShell**. Es erscheint ein blaues Fenster mit einem Eingabeprompt.

Hier gibst du Befehle ein und bestätigst mit Enter. Text aus dieser Anleitung kannst du mit Strg+C kopieren und mit **Rechtsklick** ins PowerShell-Fenster einfügen.

2.2 — Scoop installieren

Scoop ist ein Installationsverwalter. Er lädt Programme herunter, entpackt sie und macht sie im System bekannt — ohne Administratorrechte und ohne Setup-Dialoge.

Ersten Befehl eingeben:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Wenn eine Rückfrage kommt, mit `J` oder `A` bestätigen.

Dieser Befehl erlaubt PowerShell, Skripte auszuführen. Ohne ihn blockiert Windows den nächsten Schritt.

Zweiten Befehl eingeben:

```
Invoke-RestMethod -Uri https://get.scoop.sh | Invoke-Expression
```

Es läuft ein Installationsvorgang durch. Das dauert etwa eine Minute.

> **✓ Kontrolle**
>
> Gib ein:
>
> ```
> scoop --version
> ```
>
> Es muss eine Versionsnummer erscheinen.
>
> **Kommt stattdessen** *„Die Benennung 'scoop' wurde nicht als Name eines Cmdlet … erkannt"*: PowerShell-Fenster schließen, neu öffnen, noch einmal versuchen. Wenn es dann immer noch nicht geht, ist die Installation fehlgeschlagen — beide Befehle noch einmal ausführen und die Ausgabe auf Fehlermeldungen ansehen.

2.3 — AVR-Toolchain und avrdude installieren

```
scoop bucket add extras
```

```
scoop install avr-gcc avrdude make
```

Der zweite Befehl lädt mehrere hundert Megabyte herunter. Je nach Netz dauert das ein paar Minuten.

✓ Kontrolle

Schließe das PowerShell-Fenster und öffne ein neues. Das ist wichtig — sonst kennt Windows die neuen Programme noch nicht.

Dann nacheinander:

```
avr-gcc --version
avrdude -v
```

Beide müssen eine Versionsnummer ausgeben. Bei `avrdude -v` erscheint zusätzlich eine längere Meldung, das ist normal.

Kommt „wurde nicht als Name … erkannt": Das neue Fenster wurde nicht wirklich neu geöffnet, oder die Installation ist fehlgeschlagen. `scoop list` zeigt, was installiert ist.

2.4 — CLion installieren

CLion ist die Entwicklungsumgebung — das Programm, in dem du deinen Code schreibst, übersetzt und auf den Chip lädst.

1. Von https://www.jetbrains.com/clion/download/ herunterladen und installieren.
2. Beim ersten Start nach der Lizenz fragen. Als Azubi oder Student bekommst du eine kostenlose Lizenz über https://www.jetbrains.com/community/education/. Alternativ läuft CLion 30 Tage als Testversion.

Beim ersten Start fragt CLion nach Design und Tastaturbelegung. Die Voreinstellungen sind in Ordnung, du kannst alles durchklicken.

✓ Kontrolle

CLion startet und zeigt den Willkommensbildschirm mit den Punkten *New Project*, *Open* und *Get from VCS*.

Teil 3 — Programmiergerät anschließen

3.1 — Die DIP-Schalter

Der PROG-S kann mehrere Sorten von Chips programmieren: AVR, STM32, NXP. Welche Betriebsart aktiv ist, stellst du an den kleinen **DIP-Schaltern** am Gehäuse ein.

Stelle die Schalter auf AVR-ISP. Welche Kombination das ist, steht in der Herstelleranleitung:

https://www.diamex.de/dxshop/mediafiles/Sonstiges/Prog-S-Anleitung.pdf

> **ACHTUNG** **Steht das Gerät in der falschen Betriebsart, funktioniert gar nichts** — es verhält sich dann wie ein simpler USB-Seriell-Wandler und antwortet auf keine Programmierbefehle. Das ist Ursache Nummer eins bei „geht nicht".

3.2 — Stromversorgung für das Board

Der PROG-S kann das Board über die DIP-Schalter mit Strom versorgen, wahlweise 5 V oder 3,3 V.

Stelle 5 V ein. Der ATmega32 braucht mindestens 4,5 V.

Falls euer Gerät eine ältere Bauform ohne diese Möglichkeit ist, braucht das Board ein eigenes 5-V-Netzteil. Dann müssen die Massen (GND) von Netzteil und Programmiergerät verbunden sein.

3.3 — Anschließen

1. Programmiergerät per USB an den PC.
2. Flachbandkabel an die ISP-Stiftleiste des Boards, Pin-1-Markierungen beachten.

Windows 11 erkennt den PROG-S normalerweise ohne Treiberinstallation.

3.4 — COM-Port herausfinden

Windows gibt dem Gerät eine Portnummer wie COM3, COM4 oder COM7. **Diese Nummer brauchst du bei jedem einzelnen Befehl in dieser Anleitung.**

So findest du sie:

1. Rechtsklick auf das Windows-Symbol → **Geräte-Manager**
2. Abschnitt **Anschlüsse (COM & LPT)** aufklappen
3. Dort steht ein Eintrag mit einer Portnummer in Klammern

Wenn mehrere Einträge da sind und du nicht weißt, welcher es ist: Programmiergerät abziehen, schauen welcher Eintrag verschwindet, wieder einstecken.

> **Schreib dir die Nummer auf.** In dieser Anleitung steht überall `COM4` als Beispiel. Wenn bei dir COM7 steht, musst du in jedem Befehl `COM4` durch `COM7` ersetzen.
>
> Die Nummer kann sich ändern, wenn du das Gerät an einen anderen USB-Anschluss steckst. Nimm möglichst immer denselben.
>
> **✓ Kontrolle**
>
> - Im Geräte-Manager steht unter *Anschlüsse (COM & LPT)* ein Eintrag, der beim Abziehen des Programmiergeräts verschwindet.
> - Du hast die Portnummer notiert.
> - Die DIP-Schalter stehen auf AVR-ISP und 5 V.

Teil 4 — Erster Kontakt zum Controller

Bevor irgendetwas programmiert wird, prüfst du, ob PC, Programmiergerät und Chip überhaupt miteinander reden.

Dafür benutzt du `avrdude` direkt in der PowerShell. Gib ein — **COM4 durch deine Portnummer ersetzen**:

```
avrdude -c stk500v2 -P COM4 -p m32 -v
```

Die Bestandteile:

Teil	Bedeutung
`-c stk500v2`	Welches Programmiergerät. Der PROG-S spricht das STK500v2-Protokoll
`-P COM4`	An welchem Anschluss es hängt
`-p m32`	Welcher Chip programmiert wird: ATmega32
`-v`	„Erzähl ausführlich, was du tust"

Was herauskommen muss

Es rauscht einiges an Text durch. Zwei Zeilen sind wichtig:

```
Vtarget               : 5.1 V
...
Device signature = 1E 95 02 (ATmega32, ATmega32A)
```

`Vtarget` ist die gemessene Spannung am Chip. Sie muss bei etwa 5 V liegen.

`Device signature` ist die eingebaute Kennnummer des Chips. `1E 95 02` bedeutet: Es ist wirklich ein ATmega32. Die Verbindung steht.

Meldungen, die du ignorieren darfst

```
Error: unable to get parameter 0x9a
Topcard               : Unknown
```

Diese beiden erscheinen bei allen Geräten dieser Bauart. Sie sehen nach Fehlern aus, sind aber harmlos.

> **✓ Kontrolle**
>
> In der Ausgabe steht `Device signature = 1E 95 02` und `Vtarget` liegt bei etwa 5 V.
>
> **Steht dort `initialization failed`**, geh zu Teil 11 und arbeite die Liste von oben nach unten ab. Mach nicht weiter, solange das nicht geht — alles Folgende baut darauf auf.

Teil 5 — Die Fuses einstellen

Was Fuses sind

Fuses sind Grundeinstellungen, die dauerhaft im Chip stehen. Sie gehören nicht zum Programm, sondern zur Hardware-Konfiguration — vergleichbar mit dem BIOS eines PCs.

Beim ATmega32 gibt es zwei davon, die dich interessieren: das **Low Fuse Byte** (`lfuse`) und das **High Fuse Byte** (`hfuse`). Jedes ist ein Byte, also acht einzelne Schalter.

Die wichtigste Einstellung darin: **woher der Chip seinen Takt bekommt.**

Das Problem mit fabrikneuen Chips

Ein ATmega32 kommt ab Werk so eingestellt, dass er einen **eingebauten Taktgeber mit 1 MHz** benutzt. Den Quarz auf dem Board ignoriert er vollständig.

Das ist tückisch, weil das Programm trotzdem läuft — nur achtmal zu langsam. Eine LED blinkt achtmal zu träge, und die serielle Ausgabe kommt als Zeichensalat an, weil die Baudrate nicht stimmt.

Deshalb musst du bei jedem neuen Chip einmal die Fuses setzen.

Wichtig: Fuses gehören zum Chip, nicht zum Board

Die Einstellung steht im Controller selbst. Wenn du einen Chip aus dem Sockel nimmst und einen anderen einsetzt, bringt der seine eigene Konfiguration mit — auch wenn im selben Board vorher ein korrekt eingestellter Chip lief.

Merksatz: Neuer Chip im Sockel → einmal Fuses setzen.

Die richtigen Werte für eure Boards

```
lfuse = 0xFF
hfuse = 0xD9
```

Das `0x` davor bedeutet nur, dass die Zahl hexadezimal geschrieben ist.

Was dahintersteckt

Du musst das nicht auswendig können, aber einmal gesehen haben.

lfuse = 0xFF — in Bits: `1111 1111`

Bits	Name	Bedeutung bei diesem Wert
3–0	CKSEL	**Takt kommt vom externen Quarz**
5–4	SUT	Der Chip wartet nach dem Einschalten etwas, bis der Quarz stabil schwingt
6	BODEN	Unterspannungsüberwachung ausgeschaltet
7	BODLEVEL	(ohne Wirkung, solange BODEN aus ist)

Die CKSEL-Bits sind der eigentliche Punkt: Sie schalten von „eingebauter Taktgeber" auf „Quarz am Board" um.

hfuse = 0xD9 — in Bits: `1101 1001`

Bit	Name	Bedeutung bei diesem Wert
7	OCDEN	Debug-Schnittstelle aus
6	JTAGEN	**JTAG aus** — dazu gleich mehr
5	SPIEN	**Programmierung eingeschaltet — niemals ändern!**
4	CKOPT	Betriebsart des Quarzverstärkers, passend für 8 MHz
3	EESAVE	Beim Löschen wird auch das EEPROM gelöscht
2–1	BOOTSZ	(nur relevant mit Bootloader)
0	BOOTRST	**Nach dem Reset startet das Programm ganz vorn**

JTAGEN ist der Punkt, der in der Praxis die meiste Zeit kostet. Ab Werk ist JTAG eingeschaltet und belegt vier Anschlüsse des Chips (PC2 bis PC5). Wer die als normale Ein-/Ausgänge benutzen will und das nicht weiß, sucht sehr lange nach dem Fehler. Mit `0xD9` ist JTAG aus und Port C komplett frei.

> **ACHTUNG Verwechslungsgefahr:** Bei AVR-Fuses bedeutet eine **0**, dass eine Funktion **eingeschaltet** ist, und eine **1**, dass sie **aus** ist. Genau andersherum, als man es erwartet.
>
> **SPIEN niemals auf 1 setzen.** Damit schaltest du die Programmierschnittstelle ab, und der Chip lässt sich mit euren Mitteln nicht mehr retten.

Fuses setzen

```
avrdude -c stk500v2 -P COM4 -p m32 -U lfuse:w:0xff:m -U hfuse:w:0xd9:m
```

Der Teil `-U lfuse:w:0xff:m` heißt: Speicherbereich `lfuse`, Operation `w` (schreiben), Wert `0xff`, Format `m` (unmittelbarer Wert).

✓ Kontrolle

Lies die Werte zurück:

```
avrdude -c stk500v2 -P COM4 -p m32 -U lfuse:r:-:h -U hfuse:r:-:h
```

Die Ausgabe muss `0xff` und `0xd9` enthalten.

Zusätzlich muss der Befehl aus Teil 4 weiterhin funktionieren:

```
avrdude -c stk500v2 -P COM4 -p m32 -v
```

Wenn der Chip sich jetzt **nicht mehr meldet**, läuft er ohne Takt. Geh zu Teil 11, Abschnitt „Chip antwortet nach dem Fuse-Setzen nicht mehr".

Teil 6 — Das CLion-Projekt

Dieser Teil ist der ungewohnteste. Nimm dir Zeit dafür.

6.1 — Was CLion von dir erwartet

CLion baut Projekte nicht direkt, sondern über ein Zwischenwerkzeug namens **CMake**. CMake liest eine Textdatei namens `CMakeLists.txt`, in der steht, welche Quelldateien es gibt und wie sie übersetzt werden sollen. Daraus erzeugt es die eigentlichen Bauanweisungen.

Für unseren Fall gibt es eine Besonderheit. CMake ist darauf ausgelegt, Programme für **den Computer zu bauen, auf dem es läuft**. Wir wollen aber für einen ATmega bauen. Das muss man CMake ausdrücklich mitteilen — über eine zweite Datei, die **Toolchain-Datei**.

Am Ende hat dein Projekt drei Dateien:

Datei	Zweck
`main.c`	Dein Programm
`CMakeLists.txt`	Was gebaut wird und wie
`avr-toolchain.cmake`	Womit gebaut wird: avr-gcc statt dem PC-Compiler

6.2 — Projekt anlegen

1. CLion starten.
2. Im Willkommensbildschirm auf **New Project**.
3. Links in der Liste **C Executable** auswählen.
4. Bei *Location* einen Pfad eintragen, zum Beispiel: `C:\Users\DEINNAME\avr\hello` Vermeide Leerzeichen und Umlaute im Pfad. Manche der Kommandozeilenwerkzeuge kommen damit nicht zurecht.
5. Bei *Language standard* **C99** wählen.
6. Auf **Create**.

CLion legt das Projekt an und erzeugt schon zwei Dateien: eine `main.c` mit einem „Hello, World!"-Programm und eine `CMakeLists.txt`. Beide ersetzen wir gleich vollständig.

Unten im Fenster erscheint eine rote Fehlermeldung oder ein CMake-Hinweis. **Das ist an dieser Stelle normal** und verschwindet, sobald die Konfiguration stimmt.

> **✓ Kontrolle**
>
> Links im Projektbaum siehst du den Ordner `hello` mit den Dateien `main.c` und `CMakeLists.txt`.

6.3 — Die Toolchain-Datei anlegen

1. **Rechtsklick auf den Projektordner** `hello` ganz oben im Projektbaum.
2. **New → File**
3. Als Namen exakt eingeben: `avr-toolchain.cmake`
4. Enter.

Die Datei öffnet sich leer. Füge folgenden Inhalt ein:

```
# Diese Datei sagt CMake, dass wir NICHT fuer den PC bauen,
# sondern fuer einen Mikrocontroller.

set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR avr)

# Welche Compiler benutzt werden sollen
set(CMAKE_C_COMPILER   avr-gcc)
set(CMAKE_CXX_COMPILER avr-g++)

# CMake testet normalerweise, ob der Compiler ein lauffaehiges
# Programm erzeugen kann - indem es eines startet. Das geht hier
# nicht, weil das Programm fuer den ATmega ist und nicht fuer den PC.
# Diese Zeile schaltet den Test auf "Bibliothek bauen" um.
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)

# Nicht im PC-System nach Bibliotheken suchen
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

TIPP **Die Zeile mit `CMAKE_TRY_COMPILE_TARGET_TYPE` ist die wichtigste der ganzen Datei.** Ohne sie bricht CLion mit der Meldung *„C compiler is not able to compile a simple test program"* ab. Das ist der Fehler, an dem die meisten hängenbleiben.

Speichern mit **Strg+S**.

6.4 — Die CMakeLists.txt ersetzen

Öffne `CMakeLists.txt` per Doppelklick. Markiere alles (**Strg+A**) und lösche es. Füge stattdessen ein:

```
cmake_minimum_required(VERSION 3.20)
project(hello C)

# ============ HIER ANPASSEN ============
set(MCU    atmega32)        # Der Controllertyp
set(F_CPU  8000000UL)       # 8 MHz - der Quarz auf dem Board
set(PORT   COM4)            # <<< DEINE Portnummer eintragen!
set(PROG   stk500v2)        # Das Protokoll des PROG-S
set(LFUSE  0xff)
set(HFUSE  0xd9)
# ========================================

set(CMAKE_C_STANDARD 99)

# Optionen fuer den Compiler
add_compile_options(
        -mmcu=${MCU}          # Fuer welchen Chip uebersetzt wird
        -DF_CPU=${F_CPU}      # Taktfrequenz an das Programm weitergeben
        -Os                   # Auf kleine Programmgroesse optimieren
        -Wall -Wextra         # Alle Warnungen anzeigen
        -ffunction-sections -fdata-sections
)

# Optionen fuer den Linker
add_link_options(
        -mmcu=${MCU}
        -Wl,--gc-sections     # Ungenutzten Code entfernen
        -Wl,-Map=${PROJECT_NAME}.map
)

# Das eigentliche Programm
add_executable(${PROJECT_NAME}.elf main.c)
```

```
# Nach dem Uebersetzen: HEX-Datei erzeugen und Groesse anzeigen
add_custom_command(TARGET ${PROJECT_NAME}.elf POST_BUILD
        COMMAND avr-objcopy -O ihex -R .eeprom
                $<TARGET_FILE:${PROJECT_NAME}.elf> ${PROJECT_NAME}.hex
        COMMAND avr-size --format=avr --mcu=${MCU} $<TARGET_FILE:${PROJECT_NAME}.elf>
        COMMENT "HEX-Datei erzeugen"
)

# --- Zusaetzliche Knoepfe ---

add_custom_target(flash
        COMMAND avrdude -c ${PROG} -P ${PORT} -p m32
                -U flash:w:${PROJECT_NAME}.hex:i
        DEPENDS ${PROJECT_NAME}.elf
        WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
        COMMENT "Programm auf den Chip laden"
)

add_custom_target(fuses
        COMMAND avrdude -c ${PROG} -P ${PORT} -p m32
                -U lfuse:w:${LFUSE}:m -U hfuse:w:${HFUSE}:m
        COMMENT "Fuses setzen (8-MHz-Quarz)"
)

add_custom_target(readfuses
        COMMAND avrdude -c ${PROG} -P ${PORT} -p m32
                -U lfuse:r:-:h -U hfuse:r:-:h
        COMMENT "Fuses auslesen"
)

add_custom_target(testconnection
        COMMAND avrdude -c ${PROG} -P ${PORT} -p m32 -v
        COMMENT "Verbindung zum Chip pruefen"
)
```

> **WICHTIG** **Trage bei `set(PORT COM4)` deine eigene Portnummer ein.** Das ist der einzige Wert, den du wirklich anpassen musst.

Speichern mit **Strg+S**.

Was die einzelnen Blöcke tun

Block	Wirkung
`add_compile_options`	Sagt dem Compiler, für welchen Chip und mit welchem Takt
`add_executable`	Legt fest, dass aus `main.c` ein Programm wird
`add_custom_command`	Erzeugt nach dem Übersetzen die HEX-Datei und zeigt an, wie groß das Programm geworden ist
`add_custom_target`	Legt zusätzliche Schaltflächen an: Flashen, Fuses setzen, Verbindung prüfen

Die Endung `.elf` im Programmnamen ist Absicht. Ohne sie würde CMake unter Windows eine `hello.exe` erzeugen, was verwirrend wäre — die Datei lässt sich auf dem PC ja nicht starten.

6.5 — CLion die Toolchain-Datei bekannt machen

Jetzt muss CLion noch erfahren, dass es die Toolchain-Datei benutzen soll.

1. **File → Settings** (oder Strg+Alt+S)
2. Links im Baum: **Build, Execution, Deployment → CMake**
3. Im rechten Bereich gibt es ein Feld **CMake options**. Trage dort ein: `-DCMAKE_TOOLCHAIN_FILE=avr-toolchain.cmake` Das funktioniert, weil CMake relative Pfade auf den Projektordner bezieht — und dort liegt die Datei.
4. Etwas weiter unten ein Feld **Generator**. Stelle dort **Ninja** ein. (Falls Ninja nicht auswählbar ist, geht auch *MinGW Makefiles*. **Nicht** Visual Studio — das funktioniert mit avr-gcc nicht.)

5. **OK**

Falls die kurze Schreibweise nicht funktioniert

Manche CLion-Versionen legen das Build-Verzeichnis so an, dass der relative Pfad nicht mehr passt. Dann gibt es zwei Ausweichmöglichkeiten.

Variante 2 — mit CMake-Variable:

```
-DCMAKE_TOOLCHAIN_FILE=${CMAKE_SOURCE_DIR}/avr-toolchain.cmake
```

`${CMAKE_SOURCE_DIR}` ist der Projektordner. CMake setzt den Pfad selbst ein.

Variante 3 — vollständiger Pfad:

```
-DCMAKE_TOOLCHAIN_FILE=C:/Users/DEINNAME/avr/hello/avr-toolchain.cmake
```

Das geht immer, muss aber auf jedem Rechner einzeln angepasst werden.

> **ACHTUNG** **Schrägstriche nach vorn**, auch unter Windows. CMake kommt mit Rückwärtsschrägstrichen nicht zurecht — `C:\Users\...` führt zu einer Fehlermeldung.

6.6 — Projekt neu laden

Tools → CMake → Reset Cache and Reload Project

Unten öffnet sich ein Fenster mit der CMake-Ausgabe.

> **✓ Kontrolle**
>
> In der CMake-Ausgabe unten steht am Ende:
>
> ```
> -- Configuring done
> -- Generating done
> -- Build files have been written to: ...
> ```
>
> Es darf **kein** roter Text dabei sein.
>
> Oben rechts im Fenster gibt es eine Auswahlliste (das Feld neben dem grünen Pfeil). Klick sie auf. Dort müssen jetzt stehen: - `hello.elf` - `flash` - `fuses` - `readfuses` - `testconnection`
>
> **Steht dort „C compiler is not able to compile a simple test program"**: In `avr-toolchain.cmake` fehlt die Zeile `set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)` , oder CLion findet die Datei nicht. Prüfe den Pfad im Feld *CMake options* und ob die Datei wirklich `avr-toolchain.cmake` heißt (nicht `avr-toolchain.cmake.txt` — Windows blendet Endungen manchmal aus).
>
> **Steht dort „avr-gcc … not found"**: Die Toolchain ist nicht installiert oder nicht im Suchpfad. Zurück zu Teil 2.3. CLion nach der Installation neu starten, sonst kennt es den neuen Suchpfad nicht.

Teil 7 — Das erste Programm

7.1 — Wo F_CPU herkommt

Wenn du schon einmal AVR-Code gesehen hast, achte hier besonders auf einen Unterschied.

In älteren Unterlagen und in fast allen Beispielen im Internet steht die Taktfrequenz oben im Quelltext:

```
/* SO NICHT - nur zur Erklaerung, was frueher ueblich war */
#define F_CPU 8000000UL
#include <util/delay.h>
```

Bei uns steht `F_CPU` nicht mehr im Code, sondern in der `CMakeLists.txt`:

```
set(F_CPU 8000000UL)
```

Von dort gibt CMake den Wert an den Compiler weiter — über die Zeile

```
-DF_CPU=${F_CPU}
```

in den Compiler-Optionen. Das `-D` bedeutet „definiere". Der Compiler verhält sich dann so, als stünde `#define F_CPU 8000000UL` ganz oben in jeder Quelldatei.

Warum das besser ist

Die Taktfrequenz ist eine Eigenschaft der Hardware, nicht des Programms. Sie hängt davon ab, welcher Quarz auf dem Board sitzt — nicht davon, was dein Programm tut.

Sobald ein Projekt aus mehreren `.c`-Dateien besteht, wird das wichtig. Steht `#define F_CPU` in jeder Datei einzeln, musst du bei einem Hardwarewechsel jede davon anfassen. Vergisst du eine, übersetzt der Compiler ohne Fehlermeldung — die eine Datei rechnet dann mit einer anderen Taktfrequenz als der Rest. Solche Fehler zu finden ist unangenehm, weil das Programm läuft und nur die Zeiten nicht stimmen.

Mit dem Wert in der `CMakeLists.txt` gibt es **eine einzige Stelle**, die ihn festlegt. Sie gilt automatisch für alle Quelldateien.

Die Sicherung im Code

Deshalb steht ganz oben in `main.c`:

```
#ifndef F_CPU
#  error "F_CPU fehlt - kommt normalerweise aus der CMakeLists.txt"
#endif
```

Das heißt: „Wenn `F_CPU` nicht definiert ist, brich mit einer Fehlermeldung ab."

Ohne diese Zeilen würde `<util/delay.h>` stillschweigend **1 MHz** annehmen. Das Programm würde übersetzen, flashen und laufen — nur achtmal zu langsam, und die serielle Ausgabe käme als Zeichensalat an. Mit der Sicherung merkst du den Fehler sofort beim Übersetzen statt nach einer halben Stunde Fehlersuche.

> **ACHTUNG** **Schreib `#define F_CPU` nicht zusätzlich in den Quelltext.** Dann hättest du zwei Stellen, an denen derselbe Wert steht, und irgendwann unterscheiden sie sich. Der Compiler warnt in diesem Fall zwar meist, aber verlass dich nicht darauf.

7.2 — Der Quelltext

Öffne `main.c`, markiere alles (Strg+A), lösche es und füge ein:

```
/*
 * Hallo-Welt fuer ATmega32 mit 8-MHz-Quarz.
 * Sendet einmal pro Sekunde eine Textzeile ueber die serielle
 * Schnittstelle an den PC.
 */

#ifndef F_CPU
#  error "F_CPU fehlt - kommt normalerweise aus der CMakeLists.txt"
#endif

#include <avr/io.h>        /* Registernamen des ATmega32   */
#include <util/delay.h>    /* _delay_ms()                  */

#define BAUD 9600          /* Uebertragungsgeschwindigkeit */
#include <util/setbaud.h>  /* rechnet die Teilerwerte aus  */


/* Serielle Schnittstelle einschalten */
static void uart_init(void)
{
    /* Geschwindigkeit einstellen */
    UBRRH = UBRRH_VALUE;
    UBRRL = UBRRL_VALUE;
#if USE_2X
    UCSRA |=  (1 << U2X);
#else
    UCSRA &= ~(1 << U2X);
#endif

    /* Senden und Empfangen freigeben */
    UCSRB = (1 << TXEN) | (1 << RXEN);

    /* Datenformat: 8 Datenbits, keine Paritaet, 1 Stoppbit */
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
}


/* Ein einzelnes Zeichen senden */
static void uart_putc(char c)
{
    /* Warten, bis der Sendepuffer wieder frei ist */
    while (!(UCSRA & (1 << UDRE)))
        ;

    UDR = c;
}


/* Eine ganze Zeichenkette senden */
static void uart_puts(const char *s)
{
    while (*s)
        uart_putc(*s++);
}


int main(void)
{
    uart_init();

    for (;;) {
        uart_puts("Hallo Welt\r\n");
        _delay_ms(1000);
    }
}
```

Speichern mit Strg+S.

Was hier passiert

`main()` **endet nie.** Die Schleife `for (;;)` läuft endlos. Das ist bei Mikrocontrollern der Normalfall — es gibt kein Betriebssystem, zu dem das Programm zurückkehren könnte. Würde `main()` enden, stürzt der Chip in einen undefinierten Zustand.

`(1 << TXEN)` ist die übliche Schreibweise, um ein einzelnes Bit zu setzen. `TXEN` ist die Nummer des Bits, `1 << TXEN` schiebt eine Eins an diese Position. Mit `|=` wird sie ins Register geschrieben, ohne die anderen Bits zu verändern.

`while (!(UCSRA & (1 << UDRE)));` wartet, bis der Chip bereit ist, das nächste Zeichen zu senden. Ohne diese Wartezeit würden Zeichen überschrieben.

Drei Stellen, die häufig Ärger machen

`URSEL` — eine Eigenheit des ATmega32: Zwei verschiedene Register liegen auf derselben Adresse. Das Bit `URSEL` entscheidet, welches gemeint ist. Vergisst man es, landet der Wert im falschen Register und es kommt nur Zeichensalat an. Bei neueren AVR-Typen gibt es dieses Problem nicht — deshalb funktioniert Beispielcode aus dem Internet hier oft nicht.

`<util/setbaud.h>` rechnet aus `F_CPU` und `BAUD` aus, welche Zahlenwerte in die Register müssen. Dadurch stimmt die Baudrate automatisch — vorausgesetzt, `F_CPU` entspricht dem tatsächlichen Quarz.

Der `#error`-Block ganz oben ist eine Sicherung. Fehlt `F_CPU`, würde `<util/delay.h>` stillschweigend von 1 MHz ausgehen und alle Zeiten wären um Faktor 8 falsch. Der Fehler ist schwer zu finden — deshalb bricht das Programm lieber gleich beim Übersetzen ab.

7.3 — Später: derselbe Chip mit 16 MHz

Die Chips, mit denen du hier übst, werden später in einer Baugruppe eingesetzt, die mit **16 MHz** läuft. Deshalb lohnt es sich, jetzt zu verstehen, was sich dabei ändert — und was nicht.

Was sich ändert: F_CPU

In der `CMakeLists.txt`:

```
set(F_CPU 16000000UL)     # statt 8000000UL
```

Danach **Projekt neu laden, neu bauen, neu flashen.** Es genügt nicht, nur die Zeile zu ändern.

Der Quelltext bleibt dabei **unverändert**. Genau das ist der Vorteil daran, dass `F_CPU` nicht im Code steht: Ein Zahlenwert an einer Stelle, und das ganze Projekt passt sich an.

Was sich dadurch automatisch mitkorrigiert:

- `_delay_ms()` wartet weiterhin die richtige Zeit
- Die Baudrate des UART bleibt bei 9600, weil `<util/setbaud.h>` neu rechnet
- Alle Timer-Berechnungen, die du später einmal auf `F_CPU` aufbaust

Was sich nicht ändert: die Fuses

Die Fuse-Einstellung `lfuse = 0xFF`, `hfuse = 0xD9` sagt dem Chip nur: *„Nimm den Takt vom Quarz."* Sie sagt ihm nicht, **wie schnell** dieser Quarz schwingt. Das ergibt sich von selbst aus dem Bauteil auf der Platine.

Für den Bereich bis 8 MHz passt diese Einstellung. Für 16 MHz gibt es eine Feinheit:

	8 MHz	16 MHz
lfuse	`0xFF`	`0xFF`
hfuse	`0xD9`	**`0xC9`**

Der Unterschied steckt im Bit **CKOPT** (Bit 4 des hfuse). Es bestimmt, wie kräftig der eingebaute Quarzverstärker arbeitet:

- `0xD9` — CKOPT nicht gesetzt: sparsamer Betrieb, laut Datenblatt bis 8 MHz spezifiziert
- `0xC9` — CKOPT gesetzt: kräftigerer Betrieb, für 1 bis 16 MHz spezifiziert

In der Praxis läuft `0xC9` auch bei 8 MHz problemlos. Wer nur eine einzige Einstellung für beide Fälle haben will, nimmt durchgängig `0xC9`. Wir benutzen hier trotzdem `0xD9`, weil das bei 8 MHz die vom Hersteller vorgesehene Betriebsart ist.

ACHTUNG **Drei Dinge müssen zusammenpassen**, sonst läuft das Gerät falsch oder gar nicht:

1. Der **Quarz** auf der Platine
2. Der Wert **F_CPU** in der `CMakeLists.txt`
3. Die **Fuses** im Chip

Stimmt einer der drei nicht, merkst du es meist erst an der seriellen Ausgabe — dort kommt dann Zeichensalat an.

Woran du erkennst, dass etwas nicht zusammenpasst

Symptom	Ursache
Alles läuft **achtmal zu langsam**	Fuses stehen noch auf Werkseinstellung, Chip nutzt den internen 1-MHz-Taktgeber
Alles läuft **doppelt zu schnell**	`F_CPU` auf 8 MHz gesetzt, Board hat aber 16 MHz
Alles läuft **halb so schnell**	`F_CPU` auf 16 MHz gesetzt, Board hat aber 8 MHz
Serielle Ausgabe unlesbar	einer der drei Punkte oben stimmt nicht
Chip meldet sich gar nicht mehr	Fuses auf Quarz gesetzt, aber kein funktionierender Quarz vorhanden

Merksatz für später: Wenn du einen Chip aus dem Übungsboard in die 16-MHz-Baugruppe umsetzt, musst du sowohl die Fuses anpassen als auch ein mit `F_CPU = 16000000UL` gebautes Programm aufspielen.

✓ Kontrolle

Die Datei ist gespeichert. CLion zeigt bei `UBRRH`, `UCSRA` und den anderen Registernamen **keine rote Unterringelung**.

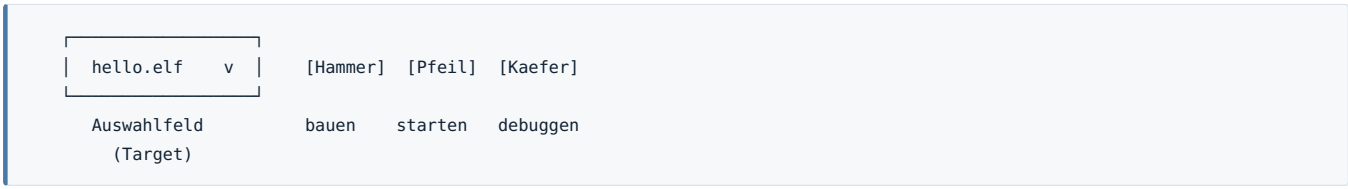
Sind sie rot: *Tools → CMake → Reset Cache and Reload Project*. CLion muss das Projekt einmal verarbeitet haben, bevor es die AVR-Registernamen kennt.

Teil 8 — Bauen

„Bauen" heißt: aus dem C-Code eine Datei machen, die der Chip versteht. Der Chip wird dabei noch nicht angefasst — er muss dafür nicht einmal angeschlossen sein.

8.1 — Die Werkzeugleiste oben rechts

Bevor du loslegst, sieh dir die Leiste oben rechts im CLion-Fenster an. Darauf kommt es jetzt an:



Das Auswahlfeld

Hier wählst du aus, **was** gemacht werden soll. Man nennt die Einträge **Targets** (Ziele). Deine `CMakeLists.txt` legt fünf davon an:

Target	Was es tut	Chip nötig?
`hello.elf`	Übersetzt den Code und erzeugt die HEX-Datei	nein
`flash`	Baut bei Bedarf neu und lädt das Programm in den Chip	**ja**
`fuses`	Schreibt die Fuse-Einstellungen	**ja**
`readfuses`	Liest die Fuses aus und zeigt sie an	**ja**
`testconnection`	Prüft, ob der Chip antwortet	**ja**

Klick das Feld an, und die Liste klappt auf. Die vier unteren Einträge sind keine Programme — es sind Befehle, die CLion für dich ausführt, damit du nicht jedes Mal in die PowerShell wechseln musst.

> **ACHTUNG** **Steht in dem Feld nur `hello.elf` und sonst nichts**, hat CMake die `CMakeLists.txt` nicht richtig gelesen. Zurück zu Teil 6.6 und das Projekt neu laden.

Hammer oder grüner Pfeil?

Das ist der Unterschied, der am Anfang für Verwirrung sorgt:

	Hammer-Symbol	Grüner Pfeil
Heißt	*Build*	*Run*
Tastenkürzel	Strg+F9	Umschalt+F10
Tut	**nur bauen**	**bauen, dann ausführen**
Beim Target `hello.elf`	übersetzt den Code	übersetzt — und versucht dann, das Programm auf dem PC zu starten. **Das geht nicht** und gibt eine Fehlermeldung
Beim Target `flash`	tut nichts Sinnvolles	baut und ruft dann avrdude auf — **das willst du**

Als Faustregel:

- Zum reinen Übersetzen: Target `hello.elf` wählen, **Hammer** drücken
- Zum Aufspielen: Target `flash` wählen, **grünen Pfeil** drücken
- Für `fuses`, `readfuses`, `testconnection`: ebenfalls **grüner Pfeil**

Der Grund: Der Hammer baut nur. Unsere Zusatz-Targets *sind* aber Befehle, die ausgeführt werden müssen — also braucht es den Pfeil.

> **TIPP** **Das Käfer-Symbol daneben ist der Debugger.** Den brauchst du hier nicht. Er würde versuchen, das Programm auf dem PC anzuhalten und schrittweise durchzugehen — das funktioniert bei AVR-Programmen ohne Zusatzhardware nicht. Finger weg, sonst bekommst du verwirrende Fehlermeldungen.

8.2 — Übersetzen

1. Im Auswahlfeld `hello.elf` wählen.
2. Auf den **Hammer** klicken (oder Strg+F9).

Unten im Fenster öffnet sich der Reiter **Build** und zeigt, was passiert.

8.3 — Was in der Ausgabe stehen muss

```
AVR Memory Usage
----------------
Device: atmega32

Program:     412 bytes (1.3% Full)
(.text + .data + .bootloader)

Data:          0 bytes (0.0% Full)
(.data + .bss + .noinit)

Build finished
```

Die genauen Zahlen können etwas abweichen. Wichtig ist: `Program:` liegt bei ein paar hundert Byte, und ganz unten steht `Build finished` ohne Fehler.

Was die Zeilen bedeuten:

- `Program` — wie viel vom 32-KB-Flash dein Programm belegt. Hier gut 400 Byte, also gut ein Prozent.
- `Data` — wie viel vom 2-KB-RAM fest belegt ist. Wird später wichtig, wenn du größere Variablen anlegst.

> **✓ Kontrolle**
>
> - Unten steht **Build finished**, kein roter Text.
> - Im Projektbaum links gibt es einen Ordner `cmake-build-debug` (oder ähnlich), darin eine Datei `hello.hex`.
>
> Falls du den Ordner nicht siehst: oben im Projektbaum das Zahnrad anklicken und *Show Excluded Files* aktivieren.
>
> **Fehler „expects a compile time integer constant" aus delay.h**: Die Optimierung fehlt. Prüfe, ob `-Os` in der `add_compile_options`-Liste steht.
>
> **Fehler „'UBRRH' undeclared"**: `-mmcu=atmega32` fehlt oder ist falsch geschrieben. Prüfe die Zeile `set(MCU atmega32)`.
>
> **Fehler „F_CPU fehlt"**: In der `CMakeLists.txt` fehlt entweder `set(F_CPU 8000000UL)` oder die Zeile `-DF_CPU=${F_CPU}` in den Compiler-Optionen.

Teil 9 — Flashen

Jetzt wandert das Programm in den Chip.

9.1 — Vorher prüfen

- Programmiergerät hängt am PC und am Board
- Chip sitzt im Sockel, Kerbe zum Stromanschluss, Hebel geschlossen
- Board hat Strom
- In der `CMakeLists.txt` steht bei `set(PORT ...)` deine Portnummer

9.2 — Ausführen

1. Im Auswahlfeld oben rechts `flash` wählen.
2. Auf den **grünen Pfeil** klicken (oder Umschalt+F10).

> **ACHTUNG** **Nicht den Hammer benutzen.** Der baut nur und ruft avrdude nicht auf. Es passiert dann scheinbar etwas, aber im Chip landet nichts. Das ist einer der häufigsten Stolpersteine.

Unten öffnet sich der Reiter **Run** mit der Ausgabe von avrdude.

CLion baut dabei automatisch vorher neu, falls du den Code geändert hast. Du musst also nicht erst den Hammer drücken und dann den Pfeil — der Pfeil allein genügt.

9.3 — Was in der Ausgabe stehen muss

```
Writing | ################################################## | 100%
Reading | ################################################## | 100%

avrdude: 412 bytes of flash verified

Avrdude done.  Thank you.
```

Die Zeile mit `verified` ist die wichtige: avrdude hat das Geschriebene zurückgelesen und mit dem Original verglichen. Steht sie da, liegt dein Programm korrekt im Chip und läuft bereits.

> **✓ Kontrolle**
>
> In der Ausgabe steht `flash verified` und am Ende `Avrdude done`.
>
> **Es passiert gar nichts oder nur „Build finished"**: Du hast den Hammer statt des grünen Pfeils benutzt, oder im Auswahlfeld steht noch `hello.elf`.
>
> **Steht dort „initialization failed"**: Siehe Teil 11.
>
> **Steht dort „verification error"**: Das Zurückgelesene stimmt nicht mit dem Geschriebenen überein. Meist ein Wackelkontakt am ISP-Stecker oder im Sockel. Nochmal versuchen; tritt es wiederholt auf, ist der Chip möglicherweise defekt.

Teil 10 — Ausgabe am PC ansehen

Das Programm läuft jetzt im Chip und sendet jede Sekunde eine Zeile. Um sie zu sehen, brauchst du den Seriell-Adapter.

10.1 — Adapter anschließen

Adapter	Board
TX	Pin 14 (PD0)
RX	Pin 15 (PD1)
GND	GND

Gekreuzt — TX des Adapters an RX des Chips und umgekehrt.

> **TIPP** Arbeitet euer Adapter mit 3,3 V statt 5 V, leg einen Widerstand von etwa 1 kΩ in die Leitung von Pin 15 zum RX des Adapters. Sonst bekommt der Adapter 5 V ab, was er nicht immer verträgt. Die Gegenrichtung ist unkritisch.

10.2 — Den zweiten COM-Port finden

Der Seriell-Adapter bekommt eine **eigene** Portnummer, die sich von der des Programmiergeräts unterscheidet.

Geräte-Manager → *Anschlüsse (COM & LPT)*. Jetzt stehen dort zwei Einträge. Der neu hinzugekommene ist der Adapter.

10.3 — Terminalprogramm

Installiere PuTTY (https://www.putty.org/) oder TeraTerm.

In PuTTY:

1. Bei *Connection type* auf **Serial** klicken
2. Bei *Serial line* die Portnummer des **Adapters** eintragen, z. B. `COM7`
3. Bei *Speed* **9600** eintragen
4. Auf **Open**

10.4 — Das Ergebnis

Im schwarzen Fenster erscheint im Sekundentakt:

```
Hallo Welt
Hallo Welt
Hallo Welt
```

> **✓ Kontrolle**
>
> Der Text erscheint lesbar und im Sekundentakt.
>
> **Es erscheint Zeichensalat** wie `ÿØÿà` oder `«¿«¿` : Die Geschwindigkeit stimmt nicht. Fast immer ist das die Fuse-Einstellung — der Chip läuft noch mit dem eingebauten 1-MHz-Taktgeber statt mit dem Quarz. Führe das Target `readfuses` aus. Steht dort `0xe1` und `0x99` , sind es die Werkseinstellungen: Target `fuses` ausführen, dann `flash` , dann nochmal probieren.
>
> **Es erscheint gar nichts**: TX und RX vertauscht, oder GND nicht verbunden, oder der falsche COM-Port. Probier zuerst, TX und RX zu tauschen.
>
> **Es kommt jede Sekunde genau ein Zeichen** oder die Ausgabe ist stark verstümmelt: ebenfalls ein Taktproblem, siehe oben.

Teil 11 — Wenn etwas nicht klappt

`initialization failed (rc = -1)`

Die Sammelmeldung für „kein Kontakt zum Chip". Arbeite diese Liste **von oben nach unten** ab und ändere immer nur **eine** Sache auf einmal.

1. Stimmt der COM-Port?

Geräte-Manager öffnen, Portnummer des Programmiergeräts prüfen. Sie ändert sich, wenn du das Gerät umsteckst. In der `CMakeLists.txt` bei `set(PORT ...)` denselben Wert eintragen und das Projekt neu laden.

2. Stehen die DIP-Schalter auf AVR-ISP?

In der falschen Betriebsart antwortet das Gerät auf keine Programmierbefehle.

3. Hat der Chip Strom?

```
avrdude -c stk500v2 -P COM4 -p m32 -v
```

In der Ausgabe nach `Vtarget` suchen.

Wert	Bedeutung
ca. **5 V**	in Ordnung
ca. **2,6 V**	**keine echte Versorgung** — diese Spannung entsteht nur durch Kriechströme über die Datenleitungen
0 V	gar keine Versorgung

Bei 2,6 V oder 0 V: DIP-Schalter für die Zielversorgung einschalten und auf 5 V stellen, oder ein externes Netzteil anschließen.

4. Sitzt der Chip richtig?

- Hebel ganz heruntergedrückt und eingerastet?
- Kerbe in der richtigen Richtung?
- Alle 40 Beine im Sockel, oder ist eines untergeklappt?

Ein einziges verbogenes Bein genügt für dieses Fehlerbild. Chip herausnehmen und die Beine gegen das Licht betrachten.

5. Sitzt das ISP-Kabel richtig?

Der 10-polige Stecker lässt sich um 180° verdreht aufsetzen. Dann sind zwei Datenleitungen vertauscht und nichts geht. Pin-1-Markierungen vergleichen. Wenn das Board eine 6-polige Leiste hat, diese benutzen.

6. Langsamer versuchen

```
avrdude -c stk500v2 -P COM4 -p m32 -B 125kHz -v
```

`-B` verlangsamt die Kommunikation. Hilft bei Chips, die mit niedrigem Takt laufen.

> **ACHTUNG** **Benutze nicht** `-F` , auch wenn avrdude es vorschlägt. Diese Option schaltet nur die Sicherheitsprüfung ab und verschleiert die eigentliche Ursache.

Chip antwortet nach dem Fuse-Setzen nicht mehr

Der Chip ist jetzt auf „Takt vom Quarz" eingestellt, bekommt aber keinen. Ohne Takt kann er nicht antworten.

Mögliche Ursachen:

- Der Chip sitzt nicht richtig im Sockel — besonders die Beine 12 und 13 prüfen, die gehen zum Quarz
- Es wurde ein falscher Fuse-Wert geschrieben
- Der Quarz auf dem Board ist defekt

Rettungsversuch: Der PROG-S kann über Pin 3 seiner 10-poligen Leiste ein Taktsignal ausgeben. Diesen Pin mit Pin 13 (XTAL1) des Chips verbinden, dann die Fuses zurücksetzen:

```
avrdude -c stk500v2 -P COM4 -p m32 -B 125kHz -U lfuse:w:0xe1:m
```

Klappt das nicht, hol jemanden dazu. Der Chip ist mit euren Mitteln nicht mehr erreichbar.

Zeichensalat im Terminal

Reihenfolge der Prüfung:

1. **Fuses:** Target `readfuses` ausführen. Muss `0xff` und `0xd9` ergeben.
2. **Baudrate im Terminalprogramm:** muss 9600 sein.
3. `F_CPU` **in der CMakeLists.txt:** muss `8000000UL` sein.

Nach jeder Änderung an Punkt 3 neu bauen und flashen.

Gar nichts im Terminal

1. TX und RX tauschen
2. GND-Verbindung prüfen
3. Richtiger COM-Port? Der Adapter hat einen anderen als das Programmiergerät
4. Läuft das Programm überhaupt? Teste mit dem Blinkprogramm weiter unten

CMake-Fehler: „C compiler is not able to compile a simple test program"

In `avr-toolchain.cmake` fehlt:

```
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)
```

Oder CLion findet die Datei nicht. Prüfe:

- Heißt die Datei wirklich `avr-toolchain.cmake`? Windows blendet bekannte Dateiendungen aus — aus `avr-toolchain.cmake` kann unbemerkt `avr-toolchain.cmake.txt` geworden sein.
- Liegt sie direkt im Projektordner, nicht in einem Unterordner?
- Stimmt der Pfad im Feld *CMake options*?

CMake-Fehler: „avr-gcc not found"

Die Toolchain ist nicht installiert oder CLion kennt den Suchpfad nicht.

1. In der PowerShell `avr-gcc --version` ausprobieren. Geht das nicht, zurück zu Teil 2.3.
2. Geht es dort: **CLion komplett schließen und neu starten.** CLion liest den Suchpfad nur beim Start ein.

Compilerfehler aus delay.h

„__builtin_avr_delay_cycles expects a compile time integer constant"

Die Optimierung ist abgeschaltet. `-Os` muss in `add_compile_options` stehen. Falls du in CLion auf ein Debug-Profil umgestellt hast, das `-O0` setzt: zurück auf die Standardeinstellung.

Nach einem Chipwechsel verhält sich alles anders

Fuses und Programm stecken **im Chip**, nicht auf dem Board. Ein frisch eingesetzter Chip ist leer und steht auf Werkseinstellung.

Nach jedem Wechsel:

1. Target `testconnection` — meldet sich der Chip?
2. Target `readfuses` — steht dort `0xff` / `0xd9`? Wenn nicht: Target `fuses`
3. Target `flash` — Programm aufspielen

Blinkprogramm als Notfalltest

Wenn du nicht weißt, ob das Problem an der seriellen Verbindung oder am Chip liegt, teste ohne UART. Ersetze `main.c` vorübergehend durch:

```
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRC |= (1 << PC0);       /* PC0 als Ausgang */

    for (;;) {
        PORTC ^= (1 << PC0);  /* umschalten */
        _delay_ms(500);
    }
}
```

LED mit einem Vorwiderstand von 330 Ω zwischen Pin 22 (PC0) und GND. Falls euer Board schon eine LED hat, den passenden Pin im Schaltplan nachsehen und im Code anpassen.

Die LED muss **einmal pro Sekunde** an- und ausgehen. Blinkt sie deutlich langsamer, läuft der Chip noch mit 1 MHz — die Fuses stimmen nicht.

Anhang A — Glossar

Begriff	Erklärung
Baudrate	Geschwindigkeit der seriellen Übertragung in Bit pro Sekunde. Sender und Empfänger müssen dieselbe eingestellt haben
Bootloader	Kleines Programm im Chip, das neue Programme über die serielle Schnittstelle annehmen kann. Benutzen wir hier nicht
Compiler	Übersetzt C-Code in Maschinencode
Cross-Compiling	Auf einem Rechner ein Programm für einen anderen Rechnertyp übersetzen
DIP-40	Bauform: 40 Beine in zwei Reihen
EEPROM	Kleiner Speicher im Chip, der ohne Strom erhalten bleibt
ELF	Dateiformat des Compilers, enthält Maschinencode plus Zusatzinformationen
F_CPU	Taktfrequenz des Chips. Steht bei uns in der `CMakeLists.txt`, nicht im Quelltext. Muss zum Quarz passen
Flash	Programmspeicher des Chips. 32 KB beim ATmega32
Flashen	Das Programm in den Flash schreiben
Fuse	Dauerhafte Grundeinstellung im Chip
HEX-Datei	Textdatei mit dem reinen Maschinencode. Das, was avrdude in den Chip schreibt
ISP	*In-System-Programming*. Programmieren, ohne den Chip auszubauen
JTAG	Debug-Schnittstelle. Belegt vier Anschlüsse, wenn sie eingeschaltet ist
Linker	Fügt die übersetzten Teile zu einem Programm zusammen
Quarz	Bauteil, das den Takt vorgibt
RAM	Arbeitsspeicher für Variablen. 2 KB beim ATmega32. Nach dem Ausschalten leer
Register	Speicherstelle im Chip, über die Hardware gesteuert wird
Reset	Neustart des Chips
Target	Auswählbarer Eintrag in CLion oben rechts. Legt fest, was beim Klick passiert
Toolchain	Die Sammlung aus Compiler, Linker und Hilfsprogrammen
UART	Serielle Schnittstelle des Chips
ZIF-Sockel	Sockel mit Hebel, aus dem sich Chips ohne Kraft entnehmen lassen

Anhang B — Befehlsübersicht

Alle Befehle für die PowerShell. `COM4` durch deine Portnummer ersetzen.

```
# Verbindung pruefen
avrdude -c stk500v2 -P COM4 -p m32 -v

# Fuses lesen
avrdude -c stk500v2 -P COM4 -p m32 -U lfuse:r:-:h -U hfuse:r:-:h

# Fuses setzen (8-MHz-Quarz)
avrdude -c stk500v2 -P COM4 -p m32 -U lfuse:w:0xff:m -U hfuse:w:0xd9:m

# Fuses setzen (16-MHz-Quarz, spaetere Baugruppe)
avrdude -c stk500v2 -P COM4 -p m32 -U lfuse:w:0xff:m -U hfuse:w:0xc9:m

# Programm aufspielen
avrdude -c stk500v2 -P COM4 -p m32 -U flash:w:hello.hex:i

# Chip komplett loeschen
avrdude -c stk500v2 -P COM4 -p m32 -e

# Langsam kommunizieren (bei Problemen)
avrdude -c stk500v2 -P COM4 -p m32 -B 125kHz -v
```

Bedeutung der Optionen

Option	Bedeutung
`-c stk500v2`	Typ des Programmiergeräts
`-P COM4`	Anschluss
`-p m32`	Chiptyp ATmega32
`-v`	Ausführliche Ausgabe
`-U`	Speicherzugriff, Aufbau: `bereich:aktion:datei:format`
`-e`	Chip löschen
`-B`	Kommunikationsgeschwindigkeit
`-n`	Nur simulieren, nichts schreiben

Wichtige Werte

	Wert
Signatur ATmega32	`1E 95 02`
Werkseinstellung Fuses	lfuse `0xE1`, hfuse `0x99`
Unsere Einstellung (8 MHz)	**lfuse `0xFF`, hfuse `0xD9`**
Spätere Baugruppe (16 MHz)	lfuse `0xFF`, hfuse `0xC9`
Taktfrequenz	8 MHz → `F_CPU = 8000000UL` in der `CMakeLists.txt`
Baudrate	9600

Anhang C — Checkliste

Zum Abhaken beim ersten Durchgang.

Vorbereitung

- ☐ Chip sitzt im Sockel, **Kerbe zum Stromanschluss**, Hebel eingerastet
- ☐ Auf dem Quarz steht 8 MHz
- ☐ Programmiergerät per USB am PC
- ☐ ISP-Kabel am Board, Pin-1-Markierungen passen
- ☐ DIP-Schalter auf AVR-ISP und 5 V

Software

- ☐ `scoop --version` gibt eine Versionsnummer aus
- ☐ `avr-gcc --version` gibt eine Versionsnummer aus
- ☐ `avrdude -v` gibt eine Versionsnummer aus
- ☐ CLion startet

Verbindung

- ☐ COM-Port des Programmiergeräts notiert: __
- ☐ `avrdude ... -v` zeigt `Device signature = 1E 95 02`
- ☐ `Vtarget` liegt bei etwa 5 V

Fuses

- ☐ Fuses geschrieben
- ☐ Zurückgelesen: `0xff` und `0xd9`
- ☐ Chip meldet sich danach immer noch

Projekt

- ☐ Drei Dateien vorhanden: `main.c`, `CMakeLists.txt`, `avr-toolchain.cmake`
- ☐ Eigene Portnummer in der `CMakeLists.txt` eingetragen
- ☐ CMake options gesetzt, Generator auf Ninja
- ☐ CMake meldet `Configuring done` / `Generating done`
- ☐ In der Target-Liste stehen `flash`, `fuses`, `readfuses`, `testconnection`

Bauen und Flashen

- ☐ Target `hello.elf` + **Hammer** → `Build finished`
- ☐ `hello.hex` existiert
- ☐ Target `flash` + **grüner Pfeil** → `flash verified`

Ausgabe

- ☐ Seriell-Adapter angeschlossen, TX und RX gekreuzt, GND verbunden
- ☐ COM-Port des Adapters notiert: __
- ☐ Terminal auf 9600 eingestellt
- ☐ „Hallo Welt" erscheint im Sekundentakt

Angaben ohne Gewähr. Maßgeblich ist das Datenblatt von Microchip zum ATmega32 / ATmega32A.